Michael Huang
EN.625.740.81
Homework 1

1.

Overfitting of polynomial matching: We have shown that the predictor defined in Equation (2.3) leads to overfitting. While this predictor seems to be very unnatural, the goal of this exercise is to show that it can be described as a thresholded polynomial. That is, show that given a training set $S = \{(x_i, f(x_i))\}_{i=1}^m \subseteq (\mathbb{R}^d \times \{0,1\})^m$, there exists a polynomial p_S such that $h_S(x) = 1$ if and only if $p_S(x) \geq 0$, where h_S is as defined in Equation (2.3). It follows that learning the class of all thresholded polynomials using the ERM rule may lead to overfitting.

The predictor defined in Equation (2.3) is the following:

$$h_S(x) = \begin{cases} y_i & \text{if } \exists i \in [m] \text{ s.t. } x_i = x \\ 0 & \text{otherwise.} \end{cases}$$

which basically just predicts based on successful training set predictions. In other words, we have that $h_S(x)$ is a perfect predictor on any point that was within the training data, and is 0 otherwise. To construct a polynomial p_S to emulate this, we need to ensure that p_S is nonnegative at every training point, and negative otherwise, as stated in the problem.

A very convenient expression that works for expressing this is in one-dimensional space is the polynomial $-(x - x_i)^2$, where we see that this is 0 when $x = x_i$, which satisfies the nonnegative requirement, and is clearly negative when $x \neq x_i$.

To extend this to d dimensions as with training set $\mathbb{R}^d$, we simply extend $x = (x_1, \ldots, x_d)$ with $x_i = (x_1 i, \ldots, x_d i)$. With d dimensions, we can still emulate the behavior as with our prior one-dimensional building block expression, i.e. we can take the sum of all elements $(x_j - x_{ij})^2$ and this will give us our 0 sum when $x = x_i$, except now in d dimensions. This is fine as a one-hot indicator of whether $x = x_i$, and we can fix the negative behavior later.

It's possible (and very likely) that we have multiple training points, so we want to indicate a hit on any of these. To do this, we can take the product of this polynomial, that is,

$$\prod(\sum_{j=1}^{d}(x_j - x_{ij})^2)$$

which returns 0 if we hit on any $x = x_i$ for some i in the training set. We then fix that this indicates a positive value for all misses on the training set by adding a negative sign, which gives us

$$\boxed{p_S = -\prod(\sum_{j=1}^{d}(x_j - x_{ij})^2)}$$

Reflecting on the last statement of the problem, we can see that the predictor, in classifying every training example correctly, has training error of 0, and therefore could be selected by ERM because there is no empirical error less than 0, so it appears that this predictor is incredibly good by the ERM rule. The clear issue that arises from the selection of this predictor is that it will perform perfectly on any point within a training set, but given some finite number of training variables, it is possible (and very likely) that the true error will be higher as it evaluates data points outside of the training set. We can therefore see that the class of thresholded polynomials from the predictor selected using the ERM rule indeed can lead to overfitting.

2.

Let H be a class of binary classifiers over a domain X. Let D be an unknown distribution over X, and let f be the target hypothesis in H. Fix some $h \in H$. Show that the expected value of $L_S(h)$ over the choice of $S|_x$ equals $L_{(D,f)}(h)$, namely,

$$\mathop{\mathbb{E}}_{S|_x \sim D^m}[L_S(h)] = L_{(D,f)}(h).$$

The training error is defined as

$$L_S(h) = \frac{|\{i \in [m] : h(x_i) \neq y_i\}|}{m}$$

and $L_{D,f}(h)$ is the true error. We can simplify this computationally by expressing success as Bernoulli variables, i.e. we count "1" for a successful classification, and "0" for an erroneous classification. We an therefore represent $L_S(h)$ as

$$L_S(h) = \frac{1}{m}\sum_{i=1}^{m}\{h(x_i) \neq f(x_i)\}$$

We take the overall expectation:

$$\mathop{\mathbb{E}}_{S|_x \sim D^m}[L_S(h)] = \mathop{\mathbb{E}}_{S|_x \sim D^m}[\frac{1}{m}\sum_{i=1}^{m}\{h(x_i) \neq f(x_i)\}]$$

By linearity of expectation:

$$= \frac{1}{m}\sum_{i=1}^{m}\mathop{\mathbb{E}}_{S|_x \sim D^m}[\{h(x_i) \neq f(x_i)\}]$$

To evaluate the expectation component, we know that each x_i follows the distribution D, so we can evaluate this expectation of a singular instance i by taking the probability essentially just taking the probability that $h(x_i) \neq f(x_i)$ at that instance i, or that

$$E\left[\{h(x_i) \neq f(x_i)\}\right] = Prob_{x_i \sim D}\big[h(x_i) \neq f(x_i)\big].$$

the latter of which we note is just the definition of the true error, $L_{D,f(h)}$. So we have that

$$\mathop{\mathbb{E}}_{S|_x \sim D^m}[L_S(h)] = \frac{1}{m}\sum_{i=1}^{m} L_{D,f(h)}$$

$$\boxed{\mathop{\mathbb{E}}_{S|_x \sim D^m}[L_S(h)] = L_{D,f(h)}}$$

exactly as we sought to find.

3.

Axis aligned rectangles: An axis aligned rectangle classifier in the plane is a classifier that assigns the value 1 to a point if and only if it is inside a certain rectangle...

1.

Let A be the algorithm that returns the smallest rectangle enclosing all positive examples in the training set. Show that A is an ERM.

Given that $A(S)$ returns the smallest rectangle enclosing all positive examples, we must have that $L_S(A(S)) = 0$, that is, the training error of the algorithm is 0. This is because the construction of A itself requires that the corresponding rectangle includes every positive training point, and all negative points therefore must lie outside the corresponding rectangle. So every negative point is outside the rectangle, and every positive point is within the rectangle, giving us a training error of 0. As this is the minimum possible training error, we can say that A fulfills the criteria to be selected using ERM.

2.

Show that if A receives a training set of size $\geq \frac{4\log(4/\delta)}{\epsilon}$ then, with probability of at least $1-\delta$ it returns a hypothesis with error of at most ϵ.

- Show that $R(S) \subseteq R^*$.
- Show that if S contains (positive) examples in all of the rectangles R_1, R_2, R_3, R_4, then the hypothesis returned by A has error of at most ϵ.
- For each $i \in \{1, \ldots, 4\}$, upper bound the probability that S does not contain an example from R_i.
- Use the union bound to conclude the argument.

We first take the suggestion of the hint to construct R^*, f, and R_1, R_2, R_3, R_4, and thus $R(S)$ accordingly. Our goal here is to essentially prove that if $m \geq \frac{4\log(4/\delta)}{\epsilon}$, then with probability of at least $1-\delta$ it gives us $L_{D,f}(A(S)) \leq \epsilon$.

To show that $R(S) \subseteq R^*$, we use the realizability assumption, with every training example being a positive exactly when it lies in R^*. The boundaries of $R(S)$ are

therefore bounded by the extremes of each positive example x_{ij}, so we know that $a_1^* \leq x_{i1} \leq b_1^*$ and $a_2^* \leq x_{i2} \leq b_2^*$. By this, we also know that $a_1^* \leq a_1 \leq b_1 \leq b_1^*$ and $a_2^* \leq a_2 \leq b_2 \leq b_2^*$, so every point in $R(S)$ must therefore be contained within R^*, which shows $R(S) \subseteq R^*$.

Next, we've constructed the regions such that the probability masses are exactly $\epsilon/4$. By definition, we know that true error of the hypothesis returned by A is equal to the proabbility mass of the part of the distribution rectangle R^* not contained by the algorithm's returned rectangel, $R(S)$. We've defined the 4 R_i such that any point in any of them would be included in one of the four learned rectangles:
- A point in R_1, the left-side rectangle, would force the algorithm's learned left boundary to be left enough to include the point.
- A point in R_2, the right-side rectangle, would force the algorithm's learned right boundary right enough to include the point.
- A point in R_3, the bottom-side rectangle, would force the algorithm's learned bottom boundary down enough to include the point.
- A point in R_4, the top-side rectangle, would force the algorithm's learned top boundary upwards enough to include the point.
We can therefore say that any part of R^* that is missing from $R(S)$ must lie in one of those constructed rectangles. We therefore have that the total error of the algorithm is the probability mass of the union of those rectangle areas, which we know to equally contain $\epsilon/4$, so with 4 rectangles that gives us an error maximized at ϵ (there might be an overlap, so the error could feasibly be lower), as we sought to show.

Next, upper bounding the probability that S does not contain an example from R_i for each $i \in \{1, \ldots, 4\}$. Given that the error probability mass of R_i is $\epsilon/4$, then one drawn example will miss with probability $1 - \frac{\epsilon}{4}$. Extrapolating this to independent draws from all 4 regions, the probability that all m samples miss is $(1 - \frac{\epsilon}{4})^m$. We use the tightest upper bound here using the knowledge that $1 + x \leq e^x$, so

equivalently $(1-x)^r \leq e^{-xr}$, so a good upper bound is therefore $e^{\frac{-\epsilon m}{4}}$.

Finally, we use the union bound to conclude the argument. From what we previously found, if S contains positive examples in all of those constructed rectangles, the error would be at most ϵ, so if the error is greater, then we must have missed at least one strip. Applying the upper bound, this would be

$$\sum_{i=1}^{4} e^{\frac{-\epsilon m}{4}} = 4e^{\frac{-\epsilon m}{4}}$$

We solve for the training set size m with the restriction of failure probability at most δ (inverting the problem statement with probability of at least $1-\delta$ with error at most ϵ):

$$\begin{aligned} 4e^{\frac{-\epsilon m}{4}} &\leq \delta \\ e^{\frac{-\epsilon m}{4}} &\leq \frac{\delta}{4} \\ \frac{-\epsilon m}{4} &\leq \log(\frac{\delta}{4}) \\ -\epsilon m &\leq 4\log(\frac{\delta}{4}) \\ m &\geq \frac{4\log(\frac{4}{\delta})}{\epsilon} \end{aligned}$$

exactly as we sought to find, which indicates to us that with at least this training set size, with probability at least $1-\delta$, we have that the true error of the algorithm is at most ϵ.

3.

Repeat the previous question for the class of axis aligned rectangles in $\mathbb{R}^d$.

To generalize to d dimensions, we have $R^* = R(a_1^*, b_1^*, \ldots, a_d^*, b_d^*)$ and a_i, b_i accordingly. We follow the same procedure.

To show that $R(S) \subseteq R^*$, we use the realizability assumption, with every training example being a positive exactly when it lies in R^*. The boundaries of $R(S)$

are therefore bounded by the extremes of each positive example x_{ij}, so we generally know that $a_j^* \leq x_{ij} \leq b_j^*$. By this, we also know that $a_j^* \leq a_j$ and $b_j \leq b_j^*$ so every point in $R(S)$ must therefore be contained within R^*, which shows $R(S) \subseteq R^*$.

We define new boundary sections according to the hint, this time instead creating $2d$ sections instead of 4. We can conceptualize this as one near each lower and upper boundary of a and b. We can think of these regions as the following:
- $R_{i,\text{lower}} = (a_1^*, b_1^*, \ldots, a_i^*, a_i, \ldots, a_d^*, b_d^*)$
- $R_{i,\text{upper}} = (a_1^*, b_1^*, \ldots, b_i, b_i^*, \ldots, a_d^*, b_d^*)$

with each one having probability mass $\epsilon/2d$. We've defined the sections such that some S containing examples in all of the sections would have learned boundaries for $R(S)$ with the learned lower bound to be to be at most a_i and the learned lower bound to be at most b_i across all d dimensions. Therefore, for any single region, we equivalently have $2d$ sections with maximum error $\frac{\epsilon}{2d}$, and with the most error possible being that without any overlap, for a maximum total error of $2d \cdot \frac{\epsilon}{2d} = \epsilon$.

Similar to before, the probability of every draw missing a single section would be $(1 - \frac{\epsilon}{2d})^m \leq e^{-\frac{\epsilon m}{2d}}$ using the property of e we used before.

Finally, taking the union bound, we get the total error and solving for failure probability at most δ being

$$\begin{aligned} 2de^{-\frac{\epsilon m}{2d}} &\leq \delta \\ e^{-\frac{\epsilon m}{2d}} &\leq \frac{\delta}{2d} \\ -\frac{\epsilon m}{2d} &\leq \log(\frac{\delta}{2d}) \\ m &\geq \frac{2d\log(\frac{2d}{\delta})}{\epsilon} \end{aligned}$$

which gets us to the same conclusion as before, generalized to d dimensions.

4.

Show that the runtime of applying the algorithm A mentioned earlier is polynomial in $d, 1/\epsilon$, and in $\log(1/\delta)$.

We start by thinking through what we do when applying A. When hitting a positive example, the algorithm must look through all d coordinates to update the corresponding minimum and maximum bounds of the slab. Processing m examples therefore takes $\mathrm{O}(md)$ time. Using the sample size bound that we previously found, we can replace m accordingly, as we have

$$m \geq \frac{2d\log(\frac{2d}{\delta})}{\epsilon}$$

so we have the runtime as

$$\mathrm{O}(\frac{2d\log(\frac{2d}{\delta})}{\epsilon} \cdot d)$$

Removing the integers which aren't relevant for runtime:

$$\mathrm{O}(\frac{d\log(\frac{d}{\delta})}{\epsilon} \cdot d)$$

$$\mathrm{O}(d^2 \cdot \frac{1}{\epsilon} \cdot \log(\frac{d}{\delta}))$$

$$\boxed{\mathrm{O}(d^2 \cdot \frac{1}{\epsilon} \cdot (\log(d) + \log(\frac{1}{\delta})))}$$

which is indeed a runtime polynomial in terms of $d, 1/\epsilon$, and in $\log(1/\delta)$.